



V-modellen

Udvikling og test af automationssystemer

Læringsmål

- ✓ Kende V-modellens struktur og de ni trin
- ✓ Forstå forskellen på *Verification* og *Validation*
- ✓ Kende koblinger mellem designtrin og testtrin
- ✓ Kunne applicere modellen på et konkret automationsprojekt
- ✓ Vide hvornår FAT, SAT og UAT hører hjemme i modellen

Indhold

1	Introduktion	2
2	Venstre side — design og specifikation	2
2.1	Requirements — kravspecifikation	2
2.2	System Design — systemdesign	3
2.3	Architecture Design — arkitekturdesign	3
2.4	Module Design — moduldesign	4
2.5	Coding — implementering	4
3	Højre side — test og verifikation	4
3.1	Unit Testing — modultest	5
3.2	Integration Testing — integrationstest	5
3.3	System Testing — systemtest	5
3.4	Acceptance Testing — accepttest	5
4	Koblinger i modellen	6
5	Et konkret eksempel	6
6	Opsummering	6

1 | Introduktion

V-modellen er en måde at strukturere udvikling og test af et automationssystem. Venstre side handler om at **definere og designe**, bunden er selve **implementeringen**, og højre side er **test og verifikation**.

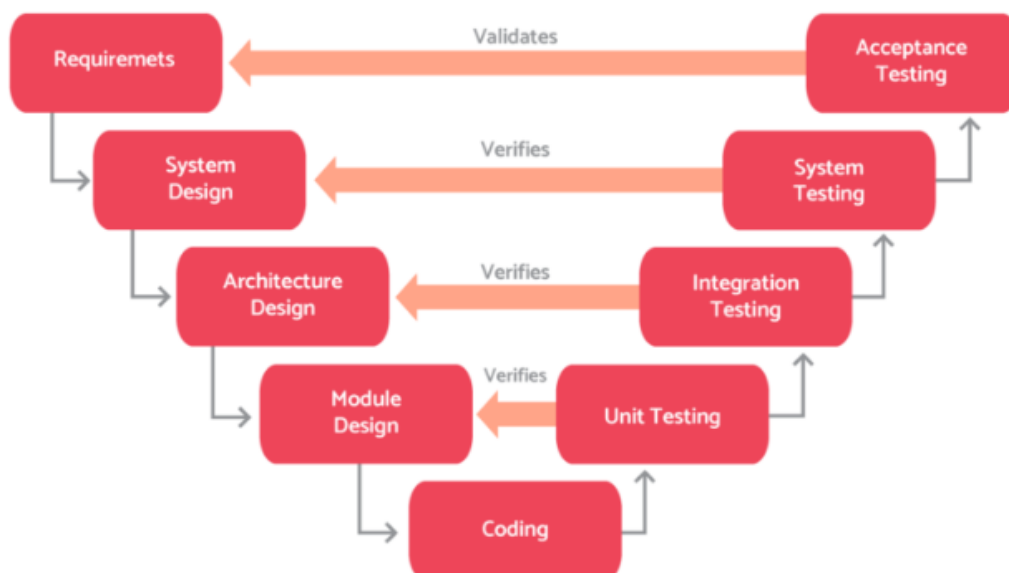
Pointen er, at hvert trin på venstre side har et modsvarende testtrin på højre side. Det er ikke tilfældigt — det er meningen.

To centrale begreber

⚡ **Verification** — bygger vi systemet rigtigt (følger vi designet)?

⚡ **Validation** — bygger vi det rigtige system (løser det opgaven)?

Det er ikke det samme, og begge dele er vigtige.



2 | Venstre side — design og specifikation

2.1 Requirements — kravspecifikation

Det første trin handler om at beskrive, hvad systemet skal kunne. Ikke *hvordan* — det kommer senere — men *hvad*.

Det er her man samler krav fra kunden, brugeren eller processen. I praksis spænder det fra ret overordnede ting til mere konkrete krav om svartider, sikkerhedsfunktioner og kommunikation.

☰ Eksempler på krav

- Anlægget skal starte og stoppe fra HMI
- En motor må ikke starte, hvis en sikkerhedsbetingelse ikke er opfyldt
- Data skal kunne trækkes fra PLC til PC
- Kommunikationsforsinkelse må ikke overstige X ms
- Anlægget skal give alarmer og kunne håndtere fejl

Man skelner gerne mellem: funktionskrav, sikkerhedskrav, driftskrav, kommunikationskrav og krav til brugerbetjening.

Det man skal have ud af fasen er en kravspecifikation — det hedder nogle gange en **URS** (User Requirements Specification). Det er det dokument, man til sidst tester imod i accepttesten.

2.2 System Design — systemdesign

Nu omsætter man kravene til en samlet teknisk løsning, men stadig på et overordnet niveau. Man beslutter hvilke hoveddele systemet består af og groft hvordan de spiller sammen — PLC, HMI, sensorer, netværk, eventuel PC eller database osv.

Man laver *ikke* kode her. Det handler om systemets struktur.

Det er her man typisk laver blokdiagrammer, netværksoversigter og beslutter, hvad der skal ligge i hvad. Fx: PLC styrer processen, HMI bruges til lokal betjening, og en Python-applikation trækker data til logging via Snap7.

☑ Output fra fasen

Systembeskrivelse, blokdiagram, hardwareoversigt, overordnet funktionsfordeling.

2.3 Architecture Design — arkitekturdesign

Her går man et niveau ned og definerer, hvordan systemets dele konkret hænger sammen. Det handler ikke bare om “PLC snakker med HMI” — man skal beskrive hvad der overføres, via hvilke interfaces og i hvilken retning.

Man fastlægger bl.a.:

- Ansvarsfordeling — hvad styres og besluttet hvor
- Interfaces og dataflow
- Kommunikationsstruktur
- Navngivning og struktur i software og data

I et Snap7-projekt er det fx her man beslutter IP-adresser, hvilke DB'er der eksponeres, og hvilke adresser og datatyper der bruges. Det er altså mere end bare “IP-konfiguration” — det er selve den tekniske struktur for samspillet.

✓ Output fra fasen

Interfacebeskrivelser, dataflow, softwarearkitektur, adresse- og tagstruktur.

2.4 Module Design — moduldesign

Her beskrives de enkelte funktioner i detaljer. Et modul kan fx være motorstyring, en ventilsekvens, temperaturregulering eller alarmhåndtering — én afgrænset funktion.

Det er på dette niveau, man designer den konkrete styringslogik:

- Sekvensforløb og state machine
- Interlocks og permissives
- Alarmer, resetlogik
- Manuelle og automatiske funktioner
- Funktionsblokke og parametre

Et motormodul kan fx have tilstandene:

Stopped → Starting → Running → Fault →
Reset

Man beslutter præcist hvilke betingelser der skal være opfyldt for at motoren må starte, og hvad der stopper den igen.

✓ Output fra fasen

Funktionsbeskrivelser, sekvensdiagrammer, state machines, FB-design.

2.5 Coding — implementering

Nu laves selve programmet og den fysiske løsning. Det kan være PLC-kode, HMI-billeder, alarmopsætning, netværkskonfiguration eller Python-kode til Snap7.

⚠ Vigtigt

Kodningen skal **følge designet**. Man bør ikke “finde på løsninger undervejs” — det giver dårlig sporbarhed og gør det sværere at teste bagefter.

✓ Output fra fasen

Færdigt PLC-program, HMI-projekt, kommunikationsopsætning, kode.

3 | Højre side — test og verifikation

Nu testes systemet trin for trin mod det der blev designet på venstre side.

3.1 Unit Testing — modultest

Her testes de enkelte moduler isoleret. Altså én funktion ad gangen:

- Virker motorblokken rigtigt?
- Skifter state machine korrekt mellem Idle, Run og Fault?
- Reagerer alarmmodulet som forventet?

I PLC-sammenhæng bruger man her simulation, forcing af signaler og kontrol af transitions. Formålet er at sikre at hver funktion virker, inden man kobler den sammen med resten.

3.2 Integration Testing — integrationstest

Her testes om modulerne virker korrekt **sammen**. Man tester nu de interfaces og sammenhænge man definerede i Architecture Design.

Eksempler

- PLC og HMI udveksler de rigtige data
- Python med Snap7 kan læse og skrive korrekte værdier i PLC
- Alarmer vises rigtigt på HMI
- Frekvensomformer reagerer korrekt på PLC-kommandoer

Formålet er at sikre at grænseflader og kommunikation fungerer i praksis.

3.3 System Testing — systemtest

Her testes hele det samlede system som én enhed — svarer til System Design-trinnet.

Man kigger nu på om hele løsningen opfører sig som beskrevet: starter anlægget korrekt, virker sekvenser i rækkefølge, fungerer betjening fra HMI, håndteres fejl og reset som planlagt?

FAT og systemtest

I automationsprojekter ligner dette typisk en **FAT** (Factory Acceptance Test), men det er vigtigt at skelne: FAT er en praktisk testform, systemtest er V-modellens faglige testniveau.

3.4 Acceptance Testing — accepttest

Her vurderes om systemet opfylder de **oprindelige krav** og kan godkendes. Accepttesten modsvarer Requirements-fasen.

Man validerer nu om systemet løser den opgave det blev bestilt til: er kravene opfyldt, kan operatøren bruge systemet, er kapacitet og svartider i orden?

I praksis hedder det **SAT** (Site Acceptance Test) eller **UAT** (User Acceptance Test), afhængigt af projektet.

4 | Koblinger i modellen

Det centrale i V-modellen er de direkte koblinger mellem venstre og højre side:

Requirements	↔	Acceptance Testing
System Design	↔	System Testing
Architecture Design	↔	Integration Testing
Module Design	↔	Unit Testing

Hvert designtrin skal altså kunne testes. Hvis man ikke kan formulere en test for et designtrin, er designet sandsynligvis ikke præcist nok.

5 | Et konkret eksempel

Opgaven

Operatøren skal kunne starte og stoppe en motor fra HMI, og en PC skal kunne læse motorstatus via Snap7.

Trin	Indhold
Krav	Start/stop fra HMI, læs motorstatus fra PC via Snap7
Systemdesign	PLC, HMI, motorstarter og PC med Python
Arkitekturdesign	PLC styrer og håndterer sikkerhedslogik. HMI sender kommandoer og viser status. Python læser DB via Snap7
Moduldesign	Motormodul: start, stop, fejl, reset, statusbits, interlock mod overlast
Kodning	PLC-program, HMI-billede og Python-script
Modultest	Test af motormodulet alene
Integrationstest	PLC–HMI og PLC–PC kommunikation
Systemtest	Hele motorstyringen som samlet funktion
Accepttest	Kan operatøren bruge løsningen, og er kravene opfyldt?

6 | Opsummering

V-modellen er en udviklings- og testmodel, hvor krav og design specificeres på venstre side, implementeres i bunden og verificeres/valideres på højre side.

✓ Hvorfor bruge V-modellen?

- Tydelig sammenhæng mellem krav, design, programmering og test
- Lettere at arbejde systematisk og dokumentere hvad der er gjort
- Tidlige fejl opdages — ikke kun til sidst ved accepttest
- Bruges bredt i industrien under navne som FAT, SAT og UAT